Plagiarism Checker -- Mentoring Guide

A Streamlit app that compares two texts and reports how similar they are, using only the Python standard library. This guide explains the two similarity measures it uses, why they can disagree, how the highlighted diff view is built, and -- importantly -- what this app can't actually detect.

1. Read this before anything else

This is not a real plagiarism detector. It only measures surface-level text overlap: shared words and shared character sequences. It will correctly flag copy-pasted or lightly-edited text. It will **not** catch:

- **Paraphrasing** -- rewording the same ideas in different words shares almost no character sequences or exact words, so both measures below score it as dissimilar.
- **Translation** -- the same content in a different language shares essentially zero surface overlap.
- **Idea plagiarism without matching wording** -- copying an argument, structure, or dataset while writing entirely original sentences.

A production-grade system needs semantic comparison -- typically sentence embeddings (turning text into vectors that capture *meaning*, not just characters) plus a large reference corpus to compare against. That's a materially bigger project; see the exercises for a pointer on how you'd start extending this one in that direction. What this app *is* good for: a clear, honest example of how far you can get with nothing but `difflib` -- and where that approach runs out.

2. Two different notions of "similar"

The app computes two independent scores and averages them, because each one catches something the other misses.

2a. Word overlap: a hand-written Jaccard index

```
def word_set(text):
    return set(re.findall(r"[a-z0-9]+", text.lower()))

def jaccard_similarity(text_a, text_b):
    words_a, words_b = word_set(text_a), word_set(text_b)
    return len(words_a & words_b) / len(words_a | words_b)
```

Each text becomes a *set* of lowercase words (word order and repetition both thrown away). The Jaccard index is |shared words| / |all words in either text| -- a classic, simple way to compare two sets. Because order doesn't matter here, two texts that say the same things in a different order can still score highly.

2b. Sequence similarity: difflib.SequenceMatcher

```
def sequence_similarity(text_a, text_b):
    return SequenceMatcher(None, text_a, text_b).ratio()
```

`SequenceMatcher` (standard library, no install needed) finds the longest matching runs of *characters* between the two texts, recursively, and reports a ratio based on how much of both texts

is covered by matches. Unlike Jaccard, this *is* sensitive to order -- rearranging the same sentences into a different sequence lowers this score even though every word is still shared.

Averaging the two gives a single number that's reasonably hard to game with just one trick (shuffling word order, or scrambling individual characters) -- though, again, neither is remotely hard to fool with real paraphrasing.

3. Highlighting the matching passages

`SequenceMatcher.get_matching_blocks()` returns the actual matching runs as `(start_in_a, start_in_b, length)` triples. The app walks Text A once, alternating between un-highlighted gaps and matched segments, and wraps each matched segment 15 characters or longer in Streamlit's `:red[...]` markdown-colouring syntax:

```
for match in matcher.get_matching_blocks():
    if match.size == 0:
        continue
    ...
    segment = text_a[match.a:match.a + match.size]
    if match.size >= 15:
        highlighted.append(f"**:red[{segment}]**")
```

The 15-character minimum matters: without it, single shared words like "the" or "is" would get highlighted individually, burying the genuinely meaningful matches (whole shared phrases or sentences) in noise.

A known rough edge, worth knowing about rather than hiding: if the pasted text itself contains markdown-special characters (*, _, #), `st.markdown` will interpret them as formatting rather than literal text, since the highlighting itself relies on real markdown syntax being injected around the matched segments. Fine for a teaching demo; a hardened version would need to escape the user's text first and use a different highlighting mechanism (e.g. raw HTML with `<mark>` tags via `st.iframe` instead of markdown).

4. Exercises

- **Ignore case and punctuation properly.** Try comparing "Hello, World!" against "hello world" -- the sequence-similarity score is dragged down by the punctuation/casing difference even though a human would call these identical. Normalize both texts (lowercase, strip punctuation) before running `SequenceMatcher`, and see how the score changes.
- **Sentence-level comparison.** Instead of comparing whole blocks of text at once, split both texts into sentences (the stdlib `re` module, splitting on `. ! ?`) and report a per-sentence-in-A similarity against the best-matching sentence in B -- much more useful for finding exactly which sentences were copied.
- **Multiple documents.** Add `st.file_uploader(accept_multiple_files=True)` and compare one document against many at once, showing a ranked table of which one it's most similar to.
- **N-gram overlap.** Instead of single-word Jaccard, build the word sets out of overlapping 3-word phrases ("n-grams") instead of single words -- a stronger signal for detecting copied phrasing specifically, rather than just shared vocabulary.

- **Semantic similarity (the real next step).** Look up "sentence embeddings" and a library like `sentence-transformers` -- turning each text into a vector and comparing vectors with cosine similarity is the actual technique that lets a checker catch paraphrasing, at the cost of a much heavier dependency and needing a pretrained model.

Written as a teaching companion to app.py -- read this guide with the code open side by side.